UD3 - Consultas SQL (V)

Nesta parte da unidade didáctica estudaremos como combinar os resultados de varias consultas mediante `UNION` e `UNION ALL`. Tamén introduciremos conceptos básicos de optimización de consultas, utilización de índices e análise dos plans de execución mediante `EXPLAIN`.

23. Combinación de seleccións con UNION e UNION ALL

23.1. Que é UNION?

O operador `UNION` permite combinar os resultados de dúas ou máis consultas `SELECT` nun único conxunto de resultados.

Cada consulta realiza unha selección independente e `UNION` combina as filas devoltas por todas elas.

```
SELECT columna1, columna2
FROM taboa1

UNION

SELECT columna1, columna2
FROM taboa2;
```

> **Idea clave:** mentres que `JOIN` combina columnas de varias táboas relacionadas, `UNION` combina filas procedentes de varias consultas.

23.2. Requisitos para utilizar UNION

Para combinar varias consultas mediante `UNION`, estas deben cumprir determinadas condicións:

- Todas as consultas deben devolver o mesmo número de columnas.
- As columnas deben aparecer na mesma orde.
- Os tipos de datos das columnas correspondentes deben ser compatibles.
- Os nomes das columnas do resultado final proceden da primeira consulta.

Non é necesario que as columnas teñan exactamente o mesmo nome, pero si deben representar información compatible.

23.3. Exemplo básico de UNION

Supoñamos que temos dúas táboas que almacenan información de persoas: unha para alumnos e outra para profesores. Queremos obter unha listaxe con todos os nomes.

```
SELECT nombre
FROM alumnos

UNION

SELECT nombre
FROM profesores;
```

O resultado contén os nomes procedentes das dúas táboas nunha única columna.

23.4. UNION elimina os duplicados

Por defecto, `UNION` elimina as filas duplicadas do resultado combinado.

Se unha mesma fila aparece nas dúas consultas, só se mostrará unha vez.

```
SELECT nombre
FROM alumnos

UNION

SELECT nombre
FROM profesores;
```

Se existe unha persoa chamada Ana nas dúas táboas, o resultado de `UNION` mostrará unha única aparición desa fila.

23.5. Exemplo con varias columnas

Podemos combinar consultas que devolvan máis dunha columna, sempre que cumpran os requisitos de compatibilidade.

```
SELECT nombre, 'Alumno' AS tipo
FROM alumnos

UNION

SELECT nombre, 'Profesor' AS tipo
FROM profesores;
```

A consulta combina os nomes e identifica a orixe de cada rexistro mediante a columna `tipo`.

Neste caso, aínda que unha persoa teña o mesmo nome nas dúas táboas, as filas non serán duplicadas porque o valor da columna `tipo` será diferente.

23.6. UNION ALL

O operador `UNION ALL` tamén combina os resultados de varias consultas, pero conserva todas as filas, incluídas as duplicadas.

```
SELECT nombre
FROM alumnos

UNION ALL

SELECT nombre
FROM profesores;
```

Se un mesmo nome aparece nas dúas táboas, aparecerá dúas veces no resultado.

23.7. Comparación entre UNION e UNION ALL

Operador	Duplicados	Comportamento
`UNION`	Elimina as filas duplicadas.	Devolve un resultado sen duplicados.
`UNION ALL`	Conserva as filas duplicadas.	Devolve todas as filas das consultas.

23.8. Compatibilidade dos tipos de datos

As columnas correspondentes deben conter tipos de datos compatibles. Por exemplo, podemos combinar dúas columnas de tipo texto ou dúas columnas numéricas.

```
-- Compatible: ambas as columnas son texto
SELECT nombre
FROM alumnos

UNION

SELECT nombre
FROM profesores;
```

```
-- Compatible: ambas as columnas son numéricas
SELECT edad AS valor
FROM alumnos

UNION

SELECT horas AS valor
FROM cursos;
```

Tamén podemos utilizar valores constantes para completar as columnas necesarias:

```
SELECT nombre, edad, 'Alumno' AS tipo
FROM alumnos

UNION ALL

SELECT nombre, NULL, 'Profesor' AS tipo
FROM profesores;
```

Neste exemplo, a segunda consulta utiliza `NULL` porque non se dispón dunha columna equivalente á idade dos alumnos.

> **Importante:** se as consultas devolven un número diferente de columnas, MariaDB producirá un erro. Tamén debemos comprobar que as columnas correspondentes teñen tipos de datos compatibles e unha orde coherente.

23.9. Combinación de máis de dúas consultas

Podemos combinar tres ou máis consultas utilizando varios operadores `UNION` ou `UNION ALL`.

```
SELECT nombre
FROM alumnos

UNION

SELECT nombre
FROM profesores

UNION

SELECT nombre
FROM clientes;
```

O resultado combina os nomes das tres táboas e elimina as filas duplicadas.

23.10. Exercicios de consolidación

1. Combina os nomes dos alumnos e dos profesores mediante `UNION`.
2. Repite a consulta utilizando `UNION ALL` e compara os resultados.
3. Combina o nome e a idade dos alumnos co nome e un valor `NULL` para os profesores.
4. Combina os nomes de alumnos, profesores e clientes nunha única consulta.
5. Combina os nomes das persoas engadindo unha columna que identifique a súa procedencia.

24. Diferenzas entre UNION, UNION ALL e outras operacións de combinación

24.1. UNION e UNION ALL

Os operadores `UNION` e `UNION ALL` permiten combinar resultados de varias consultas `SELECT`, pero teñen un comportamento diferente respecto das filas duplicadas.

`UNION` realiza a eliminación de duplicados, mentres que `UNION ALL` conserva todas as filas.

24.2. Exemplos para comparar os resultados

Supoñamos que a consulta de alumnos devolve os seguintes nomes:

```
SELECT nombre
FROM alumnos;
```

E a consulta de profesores devolve:

```
SELECT nombre
FROM profesores;
```

Se ambas as táboas conteñen o nome Ana, podemos comparar os resultados:

```
-- Elimina duplicados
SELECT nombre
FROM alumnos

UNION

SELECT nombre
FROM profesores;
```

```
-- Conserva duplicados
SELECT nombre
FROM alumnos

UNION ALL

SELECT nombre
FROM profesores;
```

24.3. Rendemento de UNION e UNION ALL

`UNION` necesita identificar e eliminar as filas duplicadas. Este proceso pode requirir operacións adicionais de ordenación ou estruturas temporais.

`UNION ALL` non necesita realizar esa eliminación, polo que normalmente ten un custo menor.

> **Recomendación:** utiliza `UNION ALL` cando necesites conservar todas as filas e non precises eliminar duplicados. Reserva `UNION` para os casos nos que realmente necesites un resultado sen repeticións.

24.4. UNION non é o mesmo que JOIN

É importante diferenciar as operacións de combinación de filas das operacións de combinación de columnas.

- `UNION` : engade filas dunha consulta ás filas doutra.
- `UNION ALL` : engade todas as filas, incluídas as duplicadas.
- `JOIN` : combina columnas de táboas relacionadas a partir dunha condición de unión.

Exemplo de `JOIN` :

```
SELECT a.nombre,
       c.nombre AS curso
FROM alumnos a
INNER JOIN cursos c
    ON a.curso_id = c.id;
```

Exemplo de `UNION` :

```
SELECT nombre
FROM alumnos

UNION

SELECT nombre
FROM profesores;
```

24.5. Outras operacións de combinación de conxuntos

Na teoría de conxuntos existen outras operacións, como a intersección e a diferenza, que permiten seleccionar os elementos comúns ou exclusivos de varios conxuntos.

- **Unión:** reúne os elementos dos dous conxuntos.
- **Intersección:** selecciona os elementos que aparecen nos dous conxuntos.
- **Diferenza:** selecciona os elementos que aparecen nun conxunto pero non no outro.

En MariaDB podemos expresar estas operacións mediante combinacións de consultas, utilizando `UNION` , `EXISTS` , `NOT EXISTS` e outras condicións. Non debemos confundir estas operacións conceptuais cos operadores SQL específicos dispoñibles noutros sistemas xestores.

24.6. Intersección mediante EXISTS

Queremos obter os nomes que aparecen tanto na táboa `alumnos` como na táboa `profesores` .

```
SELECT DISTINCT a.nombre
FROM alumnos a
WHERE EXISTS (
    SELECT *
    FROM profesores p
    WHERE p.nombre = a.nombre
);
```

A subconsulta comproba se existe un profesor co mesmo nome. `DISTINCT` evita repetir os nomes se aparecen varias veces na táboa de alumnos.

24.7. Diferenza mediante NOT EXISTS

Queremos obter os nomes dos alumnos que non aparecen na táboa de profesores.

```
SELECT DISTINCT a.nombre
FROM alumnos a
WHERE NOT EXISTS (
    SELECT *
    FROM profesores p
    WHERE p.nombre = a.nombre
);
```

A subconsulta comproba se existe un profesor co mesmo nome. Só se seleccionan os alumnos para os que non existe esa correspondencia.

24.8. Ordenación dos resultados combinados

Podemos ordenar o resultado dunha operación `UNION` ou `UNION ALL` utilizando unha única cláusula `ORDER BY` ao final da combinación.

```
SELECT nombre
FROM alumnos

UNION

SELECT nombre
FROM profesores

ORDER BY nombre ASC;
```

A ordenación aplícase ao resultado completo, non de maneira independente a cada consulta.

24.9. Ordenación por varias columnas

Cando as consultas devolven varias columnas, podemos ordenar o resultado por máis dun campo.

```
SELECT nombre, 'Alumno' AS tipo
FROM alumnos

UNION ALL

SELECT nombre, 'Profesor' AS tipo
FROM profesores

ORDER BY tipo ASC,
         nombre ASC;
```

Primeiro ordénase polo tipo de persoa e despois polo nome.

> **Importante:** a cláusula `ORDER BY` debe colocarse ao final da combinación. Os nomes das columnas do resultado son os definidos na primeira consulta. Tamén podemos utilizar a posición da columna, aínda que é preferible utilizar nomes ou alias claros.

24.10. Exercicios de consolidación

1. Combina os nomes de alumnos e profesores ordenándoos alfabeticamente.
2. Compara os resultados de `UNION` e `UNION ALL`.
3. Obtén os nomes que aparecen nas dúas táboas utilizando `EXISTS`.
4. Obtén os nomes dos alumnos que non aparecen na táboa de profesores.
5. Combina nomes e tipos de persoas, ordenando primeiro polo tipo e despois polo nome.

25. Optimización de consultas: selección de columnas, filtros e uso de índices

25.1. Que é a optimización de consultas?

A optimización de consultas consiste en mellorar a forma na que se executan as instrucións SQL para reducir o tempo de resposta e o consumo de recursos do servidor.

Unha consulta pode devolver o resultado correcto pero necesitar demasiados recursos para obtelo. A optimización busca reducir o traballo necesario para recuperar e procesar os datos.

Entre os factores que inflúen no rendemento dunha consulta destacan:

- Número de rexistros das táboas.
- Columnas recuperadas.
- Condicións de filtrado.
- Índices dispoñibles.
- Operacións de unión e agrupamento.
- Complexidade das subconsultas.

25.2. Seleccionar só as columnas necesarias

Unha das primeiras recomendacións para optimizar unha consulta é evitar recuperar columnas que non necesitamos.

Non é recomendable utilizar `SELECT *` en consultas que só precisan unha parte da información.

```
-- Consulta pouco específica
SELECT *
FROM alumnos;
```

```
-- Consulta máis específica
SELECT nombre, edad
FROM alumnos;
```

A segunda consulta recupera unicamente as columnas necesarias. Isto pode reducir a cantidade de información transferida e procesada.

25.3. Utilizar filtros para reducir os rexistros

A cláusula `WHERE` permite seleccionar os rexistros que cumpren determinadas condicións.

```
SELECT nombre, edad
FROM alumnos
WHERE edad >= 18;
```

Esta consulta recupera unicamente os alumnos maiores ou iguais a 18 anos, en lugar de devolver todos os rexistros.

Cando traballamos con táboas grandes, os filtros adecuados poden reducir considerablemente o número de filas que deben procesarse.

25.4. Evitar operacións innecesarias sobre columnas filtradas

Algunhas expresións aplicadas ás columnas dunha condición poden dificultar a utilización dun índice.

Por exemplo, supoñamos que queremos obter os alumnos cunha idade superior ou igual a 18:

```
-- Aplicación dunha operación á columna
SELECT nombre
FROM alumnos
WHERE edad + 0 >= 18;
```

É preferible expresar directamente a condición:

```
SELECT nombre
FROM alumnos
WHERE edad >= 18;
```

As expresións aplicadas á columna poden impedir ou limitar o uso eficiente dun índice convencional sobre esa columna. O comportamento exacto depende do optimizador e da expresión utilizada.

25.5. Que é un índice?

Un índice é unha estrutura de datos que permite localizar rexistros dunha táboa sen ter que revisar necesariamente todos os seus rexistros.

Os índices funcionan de maneira similar ao índice dun libro: permiten localizar información rapidamente sen ter que consultar todas as páxinas.

En MariaDB con InnoDB, os índices convencionais utilizan estruturas B-tree.

> **Idea clave:** un índice pode acelerar as consultas de busca, filtrado, ordenación e unión, pero tamén ocupa espazo e pode aumentar o custo das operacións de modificación dos datos.

25.6. Crear un índice

Podemos crear un índice sobre unha columna mediante `CREATE INDEX`.

```
CREATE INDEX idx_alumnos_edad
ON alumnos(edad);
```

Neste exemplo creamos un índice chamado `idx_alumnos_edad` sobre a columna `edad` da táboa `alumnos`.

Unha vez creado, o optimizador pode utilizalo en consultas que filtren ou ordenen por esa columna, se considera que resulta eficiente.

25.7. Consultar os índices existentes

Podemos consultar os índices dunha táboa mediante `SHOW INDEX`.

```
SHOW INDEX FROM alumnos;
```

O resultado mostra información sobre os índices, incluíndo o seu nome, as columnas que os forman e se son únicos.

25.8. Eliminar un índice

Se un índice xa non é necesario, podemos eliminalo mediante `DROP INDEX`.

```
DROP INDEX idx_alumnos_edad
ON alumnos;
```

25.9. Índices e condicións de filtrado

Os índices poden ser especialmente útiles cando realizamos buscas frecuentes sobre columnas que aparecen en condicións `WHERE`.

```
SELECT nombre, edad
FROM alumnos
WHERE edad = 20;
```

Se existe un índice sobre `edad`, o optimizador pode utilizalo para localizar os rexistros correspondentes.

Non obstante, un índice non garante sempre unha consulta máis rápida. Se a condición selecciona unha parte moi grande da táboa, pode resultar máis eficiente realizar unha lectura completa.

25.10. Índices en columnas utilizadas en JOIN

Os índices tamén son importantes cando combinamos varias táboas mediante `JOIN`.

```
SELECT a.nombre,
       c.nombre AS curso
FROM alumnos a
INNER JOIN cursos c
    ON a.curso_id = c.id;
```

Neste exemplo, o campo `cursos.id` normalmente terá un índice porque é a clave primaria. A columna `alumnos.curso_id` tamén pode beneficiarse dun índice para facilitar a localización dos alumnos asociados a cada curso.

```
CREATE INDEX idx_alumnos_curso
ON alumnos(curso_id);
```

25.11. Índices compostos

Podemos crear índices que inclúan varias columnas. Estes índices coñécense como índices compostos.

```
CREATE INDEX idx_alumnos_edad_nombre
ON alumnos(edad, nombre);
```

A orde das columnas é importante. Un índice composto sobre `(edad, nombre)` pode resultar útil para consultas que utilizan a primeira columna ou unha combinación das dúas, pero non necesariamente para consultas que utilizan só `nombre`.

25.12. O custo dos índices

Aínda que os índices poden mellorar o rendemento das consultas, tamén teñen inconvenientes:

- Ocupan espazo de almacenamento.
- Teñen que actualizarse cando se modifican os datos.
- Poden ralentizar operacións de inserción, actualización e eliminación.
- Un exceso de índices pode empeorar o rendemento global.

Por iso, os índices deben crearse en función das necesidades reais das consultas e non de maneira indiscriminada.

25.13. Exercicios de consolidación

1. Compara unha consulta que utiliza `SELECT *` con outra que selecciona unicamente dúas columnas.
2. Crea un índice sobre a columna `edad` da táboa `alumnos`.
3. Consulta os índices existentes na táboa `alumnos`.
4. Crea un índice sobre `curso_id` e analiza a súa utilidade nunha consulta con `JOIN`.
5. Crea un índice composto sobre dúas columnas e realiza consultas que utilicen esas columnas.
6. Elimina os índices creados durante a práctica.

26. Análise do plan de execución con EXPLAIN

26.1. Que é o plan de execución?

Cando executamos unha consulta SQL, o sistema xestor debe decidir como acceder aos datos e en que orde realizar as operacións necesarias.

O plan de execución describe a estratexia que o optimizador de MariaDB escolle para executar unha consulta.

A análise do plan permite identificar posibles problemas de rendemento, como lecturas completas de táboas, ausencia de índices adecuados ou operacións que procesan demasiados rexistros.

26.2. Instrución EXPLAIN

A instrución `EXPLAIN` permite consultar o plan de execución dunha consulta sen ter que executar a instrución de selección para devolver os seus resultados.

```
EXPLAIN
SELECT *
FROM alumnos
WHERE edad = 20;
```

MariaDB mostra información sobre como pretende executar a consulta, incluíndo a táboa consultada, os índices dispoñibles e o método de acceso aos rexistros.

26.3. Información principal de EXPLAIN

Entre as columnas máis importantes do resultado de `EXPLAIN` destacan:

Columna	Significado
`id`	Identificador da operación ou bloque de consulta.
`select_type`	Tipo de consulta ou subconsulta.
`table`	Táboa á que corresponde a operación.
`type`	Tipo de acceso utilizado para localizar os rexistros.
`possible_keys`	Índices que poderían utilizarse.
`key`	Índice que finalmente se utiliza, se existe.
`key_len`	Lonxitude da parte do índice utilizada.
`ref`	Columnas ou valores comparados co índice.
`rows`	Estimación do número de filas que se deben examinar.
`Extra`	Información adicional sobre a operación.

26.4. Tipos de acceso

A columna `type` indica como se accede aos rexistros da táboa. Algúns dos valores habituais son:

- `system` : a táboa contén unha única fila.
- `const` : acceso mediante unha condición que identifica unha única fila.
- `eq_ref` : busca dunha única fila mediante un índice único, normalmente nunha unión.
- `ref` : acceso mediante un índice non único.
- `range` : acceso a un intervalo de valores dun índice.
- `index` : lectura do índice completo.
- `ALL` : lectura completa da táboa.

> **Idea clave:** en xeral, unha lectura completa de táboa (`ALL`) pode ser custosa cando existen moitos rexistros. Non obstante, non sempre indica un problema: pode ser a mellor opción cando hai poucas filas ou cando a consulta necesita unha parte moi grande da táboa.

26.5. Exemplo sen índice

Supoñamos que queremos consultar os alumnos cunha idade determinada:

```
EXPLAIN
SELECT nombre, edad
FROM alumnos
WHERE edad = 20;
```

Se non existe un índice adecuado e o optimizador decide examinar todos os rexistros, podemos atopar un resultado no que a columna `type` teña o valor `ALL`.

Isto significa que MariaDB realiza unha lectura completa da táboa para comprobar que rexistros cumpren a condición.

26.6. Crear un índice e repetir a análise

Podemos crear un índice sobre a columna utilizada no filtro:

```
CREATE INDEX idx_alumnos_edad
ON alumnos(edad);
```

Agora volvemos executar `EXPLAIN`:

```
EXPLAIN
SELECT nombre, edad
FROM alumnos
WHERE edad = 20;
```

O optimizador pode utilizar o índice para localizar os rexistros correspondentes. Dependendo da distribución dos datos e do custo estimado, pode aparecer un tipo de acceso diferente, como `ref`.

Non debemos asumir que a creación dun índice obriga MariaDB a utilizalo: o optimizador escolle o plan que considera máis eficiente.

26.7. Analizar consultas con JOIN

Tamén podemos utilizar `EXPLAIN` para estudar consultas que combinan varias táboas.

```
EXPLAIN
SELECT a.nombre,
       c.nombre AS curso
FROM alumnos a
INNER JOIN cursos c
    ON a.curso_id = c.id
WHERE c.nombre = 'DAW1';
```

O resultado permite analizar:

- Que táboas participan na consulta.
- En que orde se procesan.
- Que índices se utilizan.
- Cantas filas estima o optimizador que debe examinar.

26.8. Analizar consultas con ORDER BY e GROUP BY

O plan de execución tamén pode mostrar operacións adicionais relacionadas coa ordenación e o agrupamento.

```
EXPLAIN
SELECT curso_id,
       COUNT(*) AS total
FROM alumnos
GROUP BY curso_id
ORDER BY total DESC;
```

Na columna `Extra` poden aparecer indicacións como:

- `Using where`: aplícase unha condición de filtrado.
- `Using index`: a información necesaria pode obterse mediante un índice.
- `Using temporary`: utilízase unha táboa temporal para algunha operación.
- `Using filesort`: realízase unha operación de ordenación adicional.

Estes indicadores deben interpretarse no contexto da consulta. Por exemplo, `Using temporary` ou `Using filesort` non implican automaticamente que exista un problema de rendemento.

26.9. EXPLAIN FORMAT=JSON

MariaDB permite obter información máis detallada do plan de execución mediante o formato JSON.

```
EXPLAIN FORMAT=JSON
SELECT nombre, edad
FROM alumnos
WHERE edad = 20;
```

Este formato proporciona información estruturada sobre as operacións, os custos estimados e os métodos de acceso. Resulta especialmente útil para analizar consultas complexas.

26.10. EXPLAIN ANALYZE

Ademais de `EXPLAIN`, MariaDB dispón de `ANALYZE` para executar a consulta e obter información real sobre a súa execución.

```
ANALYZE
SELECT nombre, edad
FROM alumnos
WHERE edad = 20;
```

A saída permite comparar as estimacións do optimizador cos valores observados durante a execución, como o número real de filas examinadas e os tempos de operación.

> **Precaución:** a diferenza de `EXPLAIN`, `ANALYZE` executa a consulta. Debe utilizarse con coidado en consultas custosas ou en instrucións que poidan modificar datos. Para consultas de modificación existen formas específicas de analizar a execución, polo que non se deben probar instrucións DML sen comprobar antes o seu comportamento.

26.11. Identificar consultas pouco eficientes

Ao analizar unha consulta, debemos prestar atención a diferentes sinais que poden indicar un problema:

- Lectura completa de táboas moi grandes cando se espera seleccionar poucas filas.
- Ausencia de índices nas columnas utilizadas frecuentemente en filtros ou unións.
- Estimación dun número moi elevado de filas examinadas.
- Uso innecesario de `SELECT *`.
- Operacións de ordenación ou agrupamento custosas.
- Subconsultas ou composicións que procesan máis información da necesaria.

26.12. Proceso básico de optimización

1. Identificar a consulta que presenta problemas de rendemento.
2. Executar `EXPLAIN` para coñecer o plan de execución.
3. Analizar os tipos de acceso, os índices e as filas estimadas.
4. Revisar as columnas seleccionadas e as condicións de filtrado.
5. Valorar a creación de índices adecuados.
6. Executar novamente `EXPLAIN` para comparar os plans.
7. Comprobar o rendemento real con datos representativos.

26.13. Exercicios de consolidación

1. Executa `EXPLAIN` sobre unha consulta que seleccione todos os alumnos.
2. Executa `EXPLAIN` sobre unha consulta que filtre os alumnos por idade.
3. Crea un índice sobre a columna `edad` e compara os resultados.
4. Analiza o plan dunha consulta que combine `alumnos` e `cursos` mediante `INNER JOIN`.
5. Analiza unha consulta que utilice `GROUP BY` e `ORDER BY`.
6. Compara os resultados de `EXPLAIN` e `ANALYZE` para unha consulta de selección.
7. Identifica que operacións poden representar un custo importante no plan de execución.

Resumo das operacións e optimización

Concepto	Finalidade
`UNION`	Combinar resultados eliminando filas duplicadas.
`UNION ALL`	Combinar resultados conservando todas as filas.
`JOIN`	Combinar columnas de táboas relacionadas.
`CREATE INDEX`	Crear unha estrutura para facilitar o acceso aos datos.
`SHOW INDEX`	Consultar os índices dunha táboa.
`DROP INDEX`	Eliminar un índice.
`EXPLAIN`	Consultar o plan de execución estimado.
`EXPLAIN FORMAT=JSON`	Obter información detallada do plan en formato JSON.
`ANALYZE`	Executar unha consulta e obter información real sobre a súa execución.

A combinación de consultas permite construír resultados máis completos, mentres que a optimización axuda a reducir os recursos necesarios para obtelos. A utilización de índices e a análise dos plans de execución son ferramentas fundamentais para mellorar o rendemento dunha base de datos.